

Temperature Converter Documentation

Introduction

This document provides a detailed explanation of the temperature conversion program written in Python. The program allows users to convert temperature values between Celsius (C), Fahrenheit (F), and Kelvin (K) units. It covers the basic Python concepts required to understand and implement the code.

Code Overview

The program consists of three main parts:

1. Function Definition for Temperature Conversion
2. User Input Handling
3. Conversion and Display of Result

Why Build This Project?

This project is useful for understanding:

- The implementation of mathematical formulas in Python.
- Handling user input efficiently.
- Using functions and conditionals for decision-making.
- Error handling and validation in Python programs.
- Converting between different units of temperature, which is a practical real-world application.

Prerequisites

Before starting this project, ensure you are familiar with:

- Basic Python syntax.
- Functions and conditional statements.
- Handling user input and exceptions.

Setting Up the Project

First, create a new project folder and add the following file structure:

```
/temperature-converter
|— converter.py (Main Python script)
```

Writing the Python Code

Open **converter.py** and add the following code:

1. Function Definition for Temperature Conversion

```
def convert_temperature(temp, from_unit, to_unit):
```

- **Purpose:** This function converts a temperature value from one unit to another.
- **Parameters:**
 - **temp (float):** The temperature value to be converted.
 - **from_unit (str):** The current unit of the temperature value ('C', 'F', 'K').
 - **to_unit (str):** The unit to which the temperature value will be converted ('C', 'F', 'K').

Conditional Statements

The function uses if-elif statements to check the units and perform the appropriate conversion:

```
if from_unit == 'C' and to_unit == 'F':
    return (temp * 9/5) + 32
```

Conversion Formulas:

- Celsius to Fahrenheit: $(temp * 9/5) + 32$
- Celsius to Kelvin: $temp + 273.15$
- Fahrenheit to Celsius: $(temp - 32) * 5/9$
- Fahrenheit to Kelvin: $(temp - 32) * 5/9 + 273.15$
- Kelvin to Celsius: $temp - 273.15$
- Kelvin to Fahrenheit: $(temp - 273.15) * 9/5 + 32$

- No Conversion Needed: Returns the input temperature if from_unit and to_unit are the same.
 - Invalid Input: Returns "Invalid input" if the units are not recognized.
-

2. User Input Handling

```
temp = float(input("Enter temperature value: "))  
from_unit = input("Enter current unit (C/F/K): ").upper()  
to_unit = input("Enter unit to convert to (C/F/K): ").upper()
```

- temp: Converts the user input to a floating-point number.
 - from_unit and to_unit: Converts the user input to uppercase to ensure consistency in unit comparison.
-

3. Conversion and Display of Results

Conversion

The program calls the convert_temperature function with the user-provided values:

```
result = convert_temperature(temp, from_unit, to_unit)
```

Output

The program prints the converted temperature or an error message:

```
print(f"Converted temperature: {result} {to_unit}" if  
      isinstance(result, (int, float)) else result)
```

Explanation:

- Checks if the result is a number (int or float).
 - If true, prints the converted temperature and unit.
 - If false, prints the error message returned by the function.
-

How to Run the Project

- Install Python (if not installed).
 - Open the terminal or command prompt.
 - Navigate to the project folder.
 - Run the script using:
 - python converter.py
 - Follow the on-screen prompts to input temperature values and perform conversions.
-

What You Learned

- By completing this project, you will learn:
 - Temperature Conversion Methods – Implementing Celsius, Fahrenheit, and Kelvin conversions.
 - Handling User Input – Taking and validating user input in Python.
 - Conditional Statements – Using if-elif for decision-making.
 - Error Handling – Managing incorrect input and invalid unit entries
-

Conclusion

This documentation explains how the Temperature Converter program works, covering its functions, logic, and user interactions. This project helps beginners understand Python fundamentals while providing practical hands-on experience in building an interactive application.

Happy Coding!